Smart Crop Yield Predictor

& Agentic Farm Advisory

Linear regression pipeline • LangGraph • RAG • LLM advisory

Team

Rishiwant kumar maurya Aditya Shankar
Animesh Kumar Rai Kunal Dev Sahu

Technical report

Contents

1 Executive Summary **2**

2 Dataset Overview **3**
- 2.1 Source file and schema . . . 3
- 2.2 Row filter before encoding . . . 3
- 2.3 Reference levels (`drop_first=True`) . . . 3

3 Exploratory analysis **4**

4 Preprocessing & train–test protocol **5**
- 4.1 One-hot encoding . . . 5
- 4.2 Train–test split and scaling . . . 5

5 Model training **6**
- 5.1 Model A — Linear Regression (deployed) . . . 6
- 5.2 Standardised linear coefficients . . . 6
- 5.3 Model B — Random Forest (benchmark only) . . . 6
- 5.4 Serialised artefacts . . . 8

6 Milestone 1 — deployment alignment **9**

7 Milestone 2 — agentic system architecture **10**
- 7.1 LangGraph workflow . . . 10
- 7.2 Rule-based issue detection . . . 11
- 7.3 Risk scoring vs. crop baselines . . . 12
- 7.4 RAG stack . . . 13
- 7.5 LLM advisory . . . 13

8 Milestone 1 pipeline **14**

9 Conclusion **15**

Chapter 1

Executive Summary

What this is.

- A regression model predicts `Crop_Yield_ton_per_hectare` from soil, climate, and farm inputs.
- A LangGraph agent adds rules, retrieval (RAG), and an LLM advisory layer on top of that prediction.

Data (training notebook).

- **10,000** rows × **20** columns (no missing values in the raw frame).
- After dropping `Sugarcane` and `Potato`: **6,697** rows; one-hot encoding yields **(6,697, 30)** with target `Crop_Yield_ton_per_hectare` and **29** input features.
- **80/20** split (`random_state=42`): **5,357** train / **1,340** test rows at scaled shapes $(5{,}357, 29)$ and $(1{,}340, 29)$.

Milestone 1 — model.

- **Deployed:** `LinearRegression` on `StandardScaler` features — test $R^2 =$ **0.66853**, MAE **0.8993** t/ha, RMSE **1.1252** t/ha (train $R^2 \approx 0.693$).
- **Benchmark:** `RandomForestRegressor` (`n_estimators=500`, `max_depth=6`) — lower test R^2; 5-fold CV mean $R^2 \approx$ **0.612**.
- **Saved artefacts:** `model.pkl`, `scaler.pkl`, `column.pkl` (linear model).

Milestone 2 — agent.

- **Graph:** `preprocess` → `predict` → `issue` → `risk` → `recommend` → `explain`; then `rag` → `advisory` only when risk is not Low.
- **RAG:** FAISS, `all-MiniLM-L6-v2`, top-$k = 3$ chunks.
- **LLM:** `google/flan-t5-large` via `HuggingFaceEndpoint`.

Chapter 2

Dataset Overview

2.1 Source file and schema

The Colab notebook reads:

```
df = pd.read_csv("/content/crop-yield (1).csv")
```

`df.info()` on the raw frame reports `RangeIndex: 10000 entries` and `20 columns`, all non-null. Column names:

`N`, `P`, `K`, `Soil_pH`, `Soil_Moisture`, `Soil_Type`, `Organic_Carbon`, `Temperature`, `Humidity`, `Rainfall`, `Sunlight_Hours`, `Wind_Speed`, `Region`, `Altitude`, `Season`, `Crop_Type`, `Irrigation_Type`, `Fertilizer_Used`, `Pesticide_Used`, `Crop_Yield_ton_per_hectare`.

2.2 Row filter before encoding

The notebook executes:

```
df = df[~df['Crop_Type'].isin(["Sugarcane", "Potato"])]
```

After encoding, `df_encoded.info()` shows **6,697** rows and **30** columns (14 numeric inputs + target + 15 dummy columns).

2.3 Reference levels (`drop_first=True`)

Dropped baseline categories (first level per column, lexicographic order as produced by pandas):

- `Soil_Type=Clay`, `Region=Central`, `Season=Kharif`, `Crop_Type=Cotton`, `Irrigation_Type=Canal`.

Dummy columns in the encoded frame (examples):

- `Soil_Type_Loamy`, `Soil_Type_Sandy`, `Soil_Type_Silt`
- `Region_East`, `Region_North`, `Region_South`, `Region_West`
- `Season_Rabi`, `Season_Zaid`
- `Crop_Type_Maize`, `Crop_Type_Rice`, `Crop_Type_Wheat`
- `Irrigation_Type_Drip`, `Irrigation_Type_Rainfed`, `Irrigation_Type_Sprinkler`

Chapter 3

Exploratory analysis

The notebook visualises a correlation matrix heat-map of `df_encoded`. The call uses `seaborn.heatmap` on `df_encoded.corr()`, with `cmap='RdYlGn'` and `annot=True`. Numeric heat-map cell values are not transcribed here; reproduce the figure by re-running that cell.

Chapter 4

Preprocessing & train–test protocol

4.1 One-hot encoding

```
df_encoded = pd.get_dummies(
    df,
    columns=['Soil_Type', 'Region', 'Season', 'Crop_Type', 'Irrigation_Type'],
    drop_first=True,
    dtype=int
)
```

Printed shape: **(6697, 30)**.

4.2 Train–test split and scaling

```
X = df_encoded.drop('Crop_Yield_ton_per_hectare', axis=1)
y = df_encoded['Crop_Yield_ton_per_hectare']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Observed shapes: `X_train_scaled` $(5{,}357, 29)$, `X_test_scaled` $(1{,}340, 29)$.

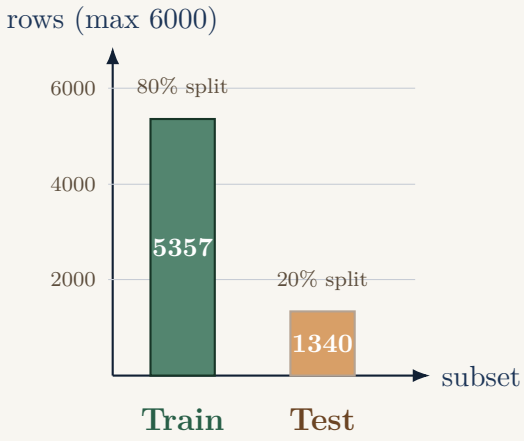

Figure 4.1: Train vs. test row counts after filtering (scaled tensor shapes from Colab).

Chapter 5

Model training

5.1 Model A — Linear Regression (deployed)

```
model = LinearRegression()
model.fit(X_train_scaled, y_train)
```

Test / train metrics (first value = test, second = train, as printed in Colab):

Metric	Test	Train
R^2	0.6685299753683484	0.6932686540237719
MAE (t/ha)	0.8992999081531897	0.8649757187284163
RMSE (t/ha)	1.125245516538983	1.0766959717537765

The generalisation gap on R^2 is $0.69327 - 0.66853 \approx$ **0.0247**.

5.2 Standardised linear coefficients

The fitted coefficients on scaled inputs are taken from a `DataFrame` built from `model.coef_` with the training column index (`X.columns`).

Feature	Coefficient
`Crop_Type_Maize`	1.155680
`Fertilizer_Used`	0.868361
`Rainfall`	0.707906
`Crop_Type_Rice`	0.850569
`Crop_Type_Wheat`	0.578425
`Soil_Moisture`	0.438410
`N`	0.246365
`Wind_Speed`	−0.257006
`K`	0.177968
`P`	0.119584
⋮ *(remaining coefficients in notebook output)*	

5.3 Model B — Random Forest (benchmark only)

```
rf_model = RandomForestRegressor(
    n_estimators=500,
```

```
    max_depth=6,
    random_state=42,
)
rf_model.fit(X_train_scaled, y_train)
```

Test / train metrics from Colab stdout:

Metric	Test	Train
R^2	0.6008194609823514	0.6699540864371375
MAE (t/ha)	0.9862918363454282	0.8963341995546024
RMSE (t/ha)	1.2348375887715302	1.1168663029762995

Five-fold CV on training data uses `cross_val_score` with `rf_model`, `X_train`, `y_train`, `cv=5`, and `scoring="r2"`. The printed mean score is **0.6118938715570774** (the notebook passes unscaled `X_train` to `cross_val_score`).

Top Gini-based importances (notebook stdout, truncated to printed rows):

Feature	Importance
`Fertilizer_Used`	0.308907
`Rainfall`	0.201344
`Crop_Type_Maize`	0.188666
`Crop_Type_Rice`	0.108097
`Crop_Type_Wheat`	0.085267
`Soil_Moisture`	0.067061
`Wind_Speed`	0.008069
`N`	0.007766
`K`	0.007375
`Soil_pH`	0.006903

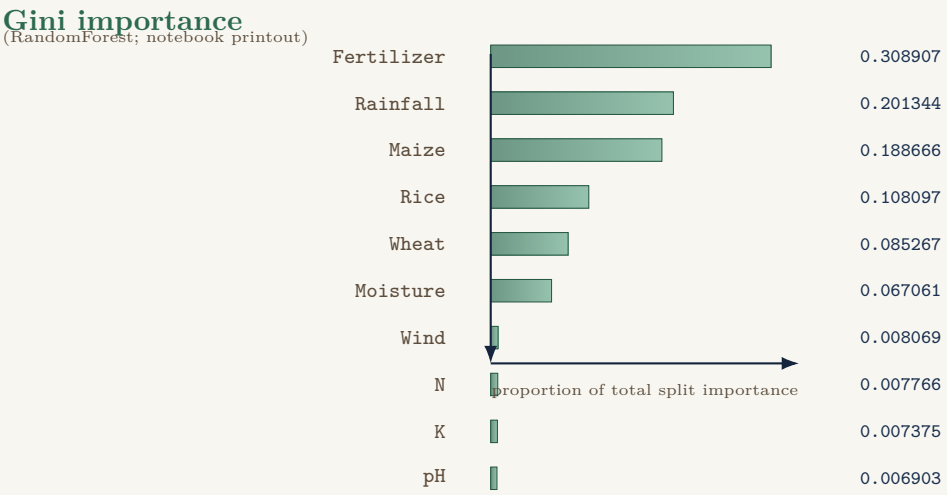

Figure 5.1: RandomForest importances (exact values from notebook stdout; row labels abbreviated).

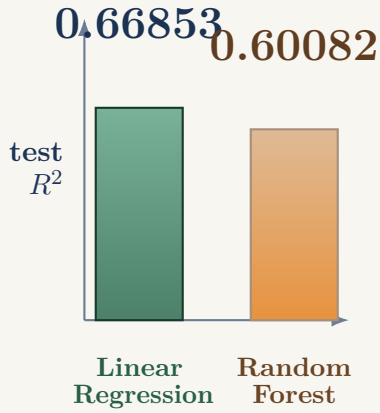

Figure 5.2: Test R^2 for $n = 1340$ hold-out rows. Full-precision metrics are in Section 5.

5.4 Serialised artefacts

The notebook saves:

```
import joblib
joblib.dump(model, "model.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(X_train.columns.tolist(), "column.pkl")
```

The loaded model class is `LinearRegression` from `sklearn.linear_model`. The scaler class is `StandardScaler` from `sklearn.preprocessing`.

Chapter 6

Milestone 1 — deployment alignment

Preprocessing for inference matches the training recipe: `get_dummies` on the same five categorical columns with `drop_first=True`, then column reindexing to the training column list.

```
import pandas as pd

def preprocess_input(user_input_dict, columns):
    input_df = pd.DataFrame([user_input_dict])

    input_encoded = pd.get_dummies(
        input_df,
        columns=['Soil_Type', 'Region', 'Season', 'Crop_Type', 'Irrigation_Type'
            ],
        drop_first=True
    )

    final_input_df = input_encoded.reindex(columns=columns, fill_value=0)

    return final_input_df
```

Notebook vs. repository minor differences.

- The training notebook passes `dtype=int` to `get_dummies`; the deployed code omits `dtype` (pandas defaults to floating one-hot columns). Values remain 0/1.
- The Streamlit UI (`app.py`) lists `Barley` and `Jute` as crops, alongside `Wheat`, `Rice`, and `Maize`.
- The encoded training frame only defines `Crop_Type_Maize`, `Crop_Type_Rice`, and `Crop_Type_Wheat` (with `Cotton` as the dropped baseline). Other crop labels therefore map with no crop-specific dummy active.

Prediction applies the scaler then `predict`:

```
def predict_yield(model, scaler, final_input_df):
    scaled_input = scaler.transform(final_input_df)
    prediction = model.predict(scaled_input)[0]
    return prediction
```

Chapter 7

Milestone 2 — agentic system architecture

7.1 LangGraph workflow

The compiled graph (`backend/graph/graph_builder.py`) defines eight nodes. Conditional routing skips RAG and the LLM when risk is `Low`.

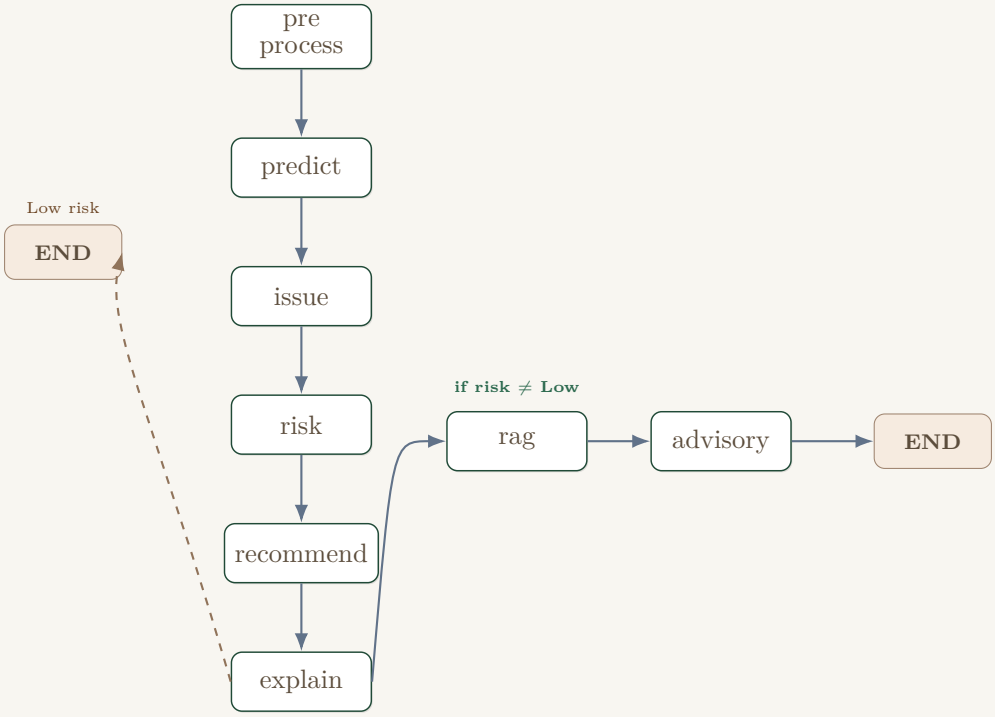

Figure 7.1: Mirrors `graph_builder.py`: linear spine then `risk_router` — dashed path skips RAG/LLM when risk is Low.

```
from langgraph.graph import StateGraph, END

from backend.core.state import FarmState
from backend.graph.nodes import get_nodes
from backend.graph.edges import risk_router


def build_graph(model, scaler, columns):
    graph = StateGraph(FarmState)
```

```
    nodes = get_nodes(model, scaler, columns)

    graph.add_node("preprocess", nodes["preprocess"])
    graph.add_node("predict", nodes["predict"])
    graph.add_node("issue", nodes["issue"])
    graph.add_node("risk", nodes["risk"])
    graph.add_node("recommend", nodes["recommend"])
    graph.add_node("explain", nodes["explain"])
    graph.add_node("rag", nodes["rag"])
    graph.add_node("advisory", nodes["advisory"])

    graph.set_entry_point("preprocess")

    graph.add_edge("preprocess", "predict")
    graph.add_edge("predict", "issue")
    graph.add_edge("issue", "risk")
    graph.add_edge("risk", "recommend")
    graph.add_edge("recommend", "explain")

    graph.add_conditional_edges(
        "explain",
        risk_router,
        {
            "skip_advisory": END,
            "do_advisory": "rag"
        }
    )

    graph.add_edge("rag", "advisory")
    graph.add_edge("advisory", END)

    return graph.compile()
```

7.2 Rule-based issue detection

Thresholds from `backend/tools/issue_tool.py`:

```
def issue_tool(state):
    data = state["input"]
    issues = []

    # Nitrogen
    if data["N"] < 50:
        issues.append({"type": "nitrogen", "severity": "high"})

    # Phosphorus
    if data["P"] < 40:
```

```
        issues.append({"type": "phosphorus", "severity": "high"})

    # Potassium
    if data["K"] < 40:
        issues.append({"type": "potassium", "severity": "medium"})

    # Rainfall
    if data["Rainfall"] < 500:
        issues.append({"type": "rainfall", "severity": "medium"})

    # Organic Carbon
    if data["Organic_Carbon"] < 0.5:
        issues.append({"type": "soil_fertility", "severity": "medium"})

    # Soil pH
    if data["Soil_pH"] < 5.5:
        issues.append({"type": "soil_acidic", "severity": "medium"})
    elif data["Soil_pH"] > 7.5:
        issues.append({"type": "soil_alkaline", "severity": "medium"})

    # Temperature
    if data["Temperature"] > 38:
        issues.append({"type": "heat_stress", "severity": "high"})

    state["issues"] = issues
    return state
```

7.3 Risk scoring vs. crop baselines

`backend/tools/risk_tool.py` compares predicted yield to fixed baselines (Wheat 3.5, Rice 4.0, Maize 3.0 t/ha; default 3.0 for other crops) using ratio bands:

```
def risk_tool(state):
    crop = state["input"]["Crop_Type"]
    pred = state["prediction"]

    baseline = {
        "Wheat": 3.5,
        "Rice": 4.0,
        "Maize": 3.0
    }

    base = baseline.get(crop, 3.0)
    ratio = pred / base

    if ratio < 0.6:
        risk = "High"
```

```
    elif ratio < 0.85:
        risk = "Moderate"
    else:
        risk = "Low"

    state["risk"] = risk
    return state
```

7.4 RAG stack

Vector store (`backend/rag/vector_store.py`):

- FAISS index persisted under `faiss_index/`.
- Embeddings: `sentence-transformers/all-MiniLM-L6-v2`.
- Text splitting: `CharacterTextSplitter` with `chunk_size=600` and `chunk_overlap=80` on `backend/rag/knowledge_base.txt`.

Retriever (`backend/rag/retriever.py`): `similarity_search` with `k=3`, plus an LRU cache with `maxsize=50`.

7.5 LLM advisory

Module `backend/llm/advisor.py`:

- Uses `HuggingFaceEndpoint` with `repo_id="google/flan-t5-large"` and `temperature=0.3`.
- Parses JSON when the response is valid; otherwise wraps free text in a small fallback structure.

Chapter 8

Milestone 1 pipeline

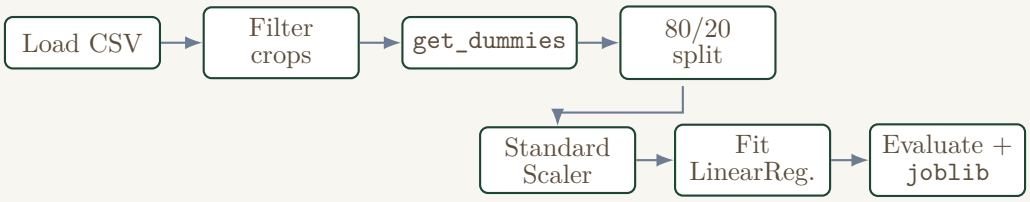

Figure 8.1: End-to-end Milestone 1 flow as implemented in Colab (orthogonal routing avoids crossing edges).

Chapter 9

Conclusion

The Colab notebook trains and evaluates both a linear model and a regularised shallow random forest; on the recorded 80/20 split the linear model achieves strictly better test R^2 and lower test MAE/RMSE. The persisted artefacts correspond to **LinearRegression**. The application codebase consumes those artefacts and adds an auditable agentic layer: deterministic rules, optional retrieval-grounded context, and an LLM JSON advisory path for non-low risk outcomes.